

C#-da yield nədir?

yield return metodun işini tam bitirmədən bir nəticə verməsinə, sonra isə növbəti nəticə istəyəndə qaldığı yerdən davam etməsinə imkan verir.

Bir cümləlik yaddaş modeli

Adi return deyir: "nəticəni verdim, metod bitdi". yield return deyir: "bu nəticəni götür, mən burada dayanım; növbəti istəyəndə davam edərim".

Gündəlik bənzətmə

Bir anbarda qutular axtardığını düşün. Adi List yanaşması əvvəlcə bütün uyğun qutuları tapır, bir yerə yığır, sonra sənə verir. yield isə uyğun qutu tapdıqca birini verir və növbəti qutu tələb olunanadək axtarışı saxlayır.

List qaytaran metod	yield qaytaran metod
Bütün nəticələri əvvəlcədən hazırlayır.	Nəticələri tələb olunduqca hazırlayır.
Aralıq List yaddaşda saxlanılır.	Əlavə List yaratmaq məcburi deyil.
Metod return zamanı bitir.	Metodun vəziyyəti saxlanılır və sonra davam edir.

Əsas fikir

yield nəticələrin saxlandığı hazır kolleksiya deyil. O, nəticələrin necə tapılacağını göstərən iterator yaradır.

return və yield return fərqi

Adi return istifadə edən metod əvvəlcə nəticələrin hamısını hazırlayır:

```
IEnumerable<int> GetEvenNumbers()
{
    var result = new List<int>();

    for (var number = 1; number <= 6; number++)
    {
        if (number % 2 == 0)
            result.Add(number);
    }

    return result;
}
```

Eyni məntiq yield return ilə belə yazılır:

```
IEnumerable<int> GetEvenNumbers()
{
    for (var number = 1; number <= 6; number++)
    {
        if (number % 2 == 0)
            yield return number;
    }
}
```

Kod nə vaxt işləyir?

```
var numbers = GetEvenNumbers(); // Metodun gövdəsi hələ işləmir.

foreach (var number in numbers) // Burada nəticələr istənilir.
{
    Console.WriteLine(number);
}
```

Deferred execution - gecikdirilmiş icra

GetEvenNumbers() çağırışı iterator obyektini qaytarır. Dövr yalnız foreach, ToList(), First(), Any() kimi enumeration əməliyyatı başlayanda işləyir.

yield addım-addım necə işləyir?

1

Enumeration başlayır

foreach iterator-dan ilk nəticəni istəyir. Arxa planda MoveNext() çağırılır.

2

Metod işləməyə başlayır

Kod ilk yield return sətirinə qədər icra olunur.

3

Bir nəticə qaytarılır

Tapılan dəyər foreach-ə verilir, amma metod tamamlanmır.

4

Vəziyyət yadda saxlanılır

Dövrün indeksi, lokal dəyişənlər və dayanılan sətir iterator tərəfindən saxlanılır.

5

Növbəti nəticə tələb olunur

foreach növbəti MoveNext() çağıranda metod yield return-dan sonrakı sətirdən davam edir.

6

Metod tamamlanır

Kodun sonuna çatanda və ya yield break işləyəndə daha nəticə olmadığı bildirilir.

Yadda saxlanan vəziyyət

Çağırış	Metodun davranışı	Nəticə
1-ci MoveNext()	1-dən başlayır, 2-də dayanır	2
2-ci MoveNext()	2-dən sonra davam edir, 4-də dayanır	4
3-cü MoveNext()	4-dən sonra davam edir, 6-da dayanır	6
4-cü MoveNext()	Metodun sonuna çatır	Nəticə yoxdur

yield return metodu bağlamır

Ən çox qarışdırılan nöqtə budur: yield return işləyəndə metodun icrası dayandırılır, amma iterator üçün metodun vəziyyəti yaşayır. Növbəti nəticə istəyəndə həmin vəziyyətdən davam edir.

Sizin calculator-da yield

BuildCandidateIntervals metodu bir iş saati üçün maksimum iki başlanğıc tarixini yoxlayır: dünən və bu gün. Uyğun interval tapdıqca yield return ilə SelectMany-ə verir.

```
private static IEnumerable<WorkingHourInterval> BuildCandidateIntervals(
    RestaurantWorkingHour workingHour,
    DateTimeOffset localDateTime)
{
    var currentDate = DateOnly.FromDateTime(localDateTime.DateTime);

    foreach (var startDate in new[] { currentDate.AddDays(-1), currentDate })
    {
        if (startDate.DayOfWeek != workingHour.DayOfWeek)
            continue;

        var start = new DateTimeOffset(
            startDate.ToDateTime(workingHour.OpensAt),
            localDateTime.Offset);

        var endDate = startDate.AddDays(workingHour.CloseDayOffset);
        var end = new DateTimeOffset(
            endDate.ToDateTime(workingHour.ClosesAt),
            localDateTime.Offset);

        if (end > start)
            yield return new WorkingHourInterval(start, end);
    }
}
```

Real ssenari

Məlumat	Dəyər
İş günü	Bazar ertəsi
İş saati	18:00 - 02:00
CloseDayOffset	1 - növbəti gün
Yoxlanan vaxt	Çərşənbə axşamı, 00:30

Dünənki tarix Bazar ertəsidir, ona görə interval qurulur: Bazar ertəsi 18:00-dan Çərşənbə axşamı 02:00-a qədər. end > start olduğu üçün interval yield return ilə qaytarılır. Bu günün tarixi Çərşənbə axşamıdır və workingHour Bazar ertəsinə aid olduğuna görə continue ilə keçilir.

Bu kodda yield-in real faydası

Maksimum iki tarix yoxlandığı üçün böyük performans qazancı yoxdur. Əsas üstünlük metodun məqsədini təmiz göstərməsidir: uyğun interval tapdıqca onu qaytar.

Nə vaxt yield istifadə etmək yaxşıdır?

yield xüsusilə bu hallarda uyğundur:

1

Nəticələr ardıcıl hesablanır

Bütün nəticələri əvvəlcədən hazırlamaq məcburi deyil.

2

Məlumat böyük ola bilər

Minlərlə və ya milyonlarla elementi əlavə List-də saxlamaq istəmərsən.

3

İlk nəticə kifayət edə bilər

First(), Any() və Take() bütün ardıcılığı hesablamaqdan əvvəl dayana bilər.

4

Iterator kodunu sadələşdirmək istəyirsən

Əl ilə IEnumerator state machine yazmaq əvəzinə compiler onu yaradır.

Diqqət edilməli məqamlar

Mövzu	Nə baş verir?
Təkrar enumeration	Eyni IEnumerable iki dəfə oxunsa, metod adətən iki dəfə yenidən işləyir.
Exception vaxtı	Xəta metod çağırılan anda deyil, enumeration başlayanda yarana bilər.
Yan təsirlər	Log yazmaq və data dəyişmək kimi əməliyyatlar nəticə oxunduqca baş verir.
ToList()	Bütün iteratoru dərhal sona qədər işlədir və nəticələri yaddaşa yığır.
yield break	Iteratoru dərhal bitirir və daha nəticə qaytarılmır.

Son yaddaş cümləsi

yield return = bir nəticə ver, mövqeyi yadda saxla, növbəti tələb gələndə davam et.

Özünü yoxla

BuildCandidateIntervals iki uyğun interval qaytarsa, SelectMany onları bir axına çevirir. Sonrakı Where yalnız localDateTime daxilində aktiv olanları saxlayır. Yeni yield intervalı seçmir; sadəcə mümkün intervalı axına verir. Son seçim TryGetActiveInterval metodunda edilir.